IS 1210
Database Systems

Lecturer 2 – Normalization (Part 2)

MVPT Lakshika,

Lecturer (Probationary),

Department of Information Systems Engineering, UCSC.

Contact: tlv@ucsc.cmb.ac.lk

Outline of this lecture

1. Key types
2. Functional Dependency
3. Process of Normalization

1. Key types

Key Types

Super key

- A super key is an attribute, or set of attributes, that identifies tuples of a relation uniquely, while a candidate key is a minimal super key.

Candidate Key

- A candidate key is a minimal set of attributes of an entity that uniquely identifies each occurrence of that entity.
- You may identify more than one candidate key, in which case you must choose one to be the **primary key**; the remaining candidate keys are called **alternate keys**.

Example 1**:** (studentID, studentRollNo, Address, Marks)

Candidate keys: studentID and studentRollNo. If the **primary key** is studentID and then studentRollNo is considered an **alternate key.**

Key Types

Candidate Key (Cont…)

Example 2: Is sName a suitable candidate key for the *Staff* entity?

- Two people with the same name can join *the staff*, which would clearly invalidate the choice of name as a candidate key.
- In such cases, rather than coming up with combinations of attributes that may provide uniqueness, it may be better to use an existing attribute that would always ensure uniqueness (staffNo) for the Staff entity.
- **Or else**, define a new attribute that would provide uniqueness.

Key Types

Primary Key

- This is a candidate key that is selected to identify tuples uniquely within the relation.
- The value of the primary key cannot be repeated inside a table and cannot hold null values.
- This is the minimal identifier that is used to identify tuples uniquely.
- Generally first column is defined as the primary key.
- The candidate keys that are not selected to be the primary key are called **alternate keys**.

Example: The staffNo and National Identity Card (NIC) number can be considered as the two candidate keys for the *Staff* entity. Which one must be selected as the primary key?

- Consider the attribute length, the minimal number of attributes required, and the future certainty of uniqueness.

Key Types

Composite primary key

- A candidate key that consists of two or more attributes.

Example: Consider the composite primary key of the *Viewing* relation, comprising the client number (clientNo) and the property number (propertyNo).

Viewing (clientNo, propertyNo, viewDate, comment)

- Users should not be able to insert a tuple into the *Viewing* relation with either a *null* for the clientNo attribute, or a *null* for the propertyNo attribute, or *nulls* for both attributes.

Key Types

Foreign Key

- A foreign key is an attribute, or set of attributes, within one relation that is the candidate key of another relation.
- This key can have repetitive values (but to uniquely identify each entry, the table can still have a primary key column separate from a foreign key column).
- The foreign key will create a relation with another table where those values are defined as primary keys.

Example: The ownerNo attribute is a primary key within the *Owner* relation and is also present within the *PropertyForRent* relation as a foreign key.

PropertyForRent	(propertyNo, pAddress, rent, ownerNo)
Owner	(ownerNo, oName)

Key Types

Compound Key

- A compound key is a specific type of composite key where two or more columns are combined to uniquely identify a record, and each of those columns is also a foreign key referencing another table.
- A subset of a composite key where every single column within the key is a foreign key to a primary key in another table.
- This often occurs in a junction table that resolves a many-to-many relationship.

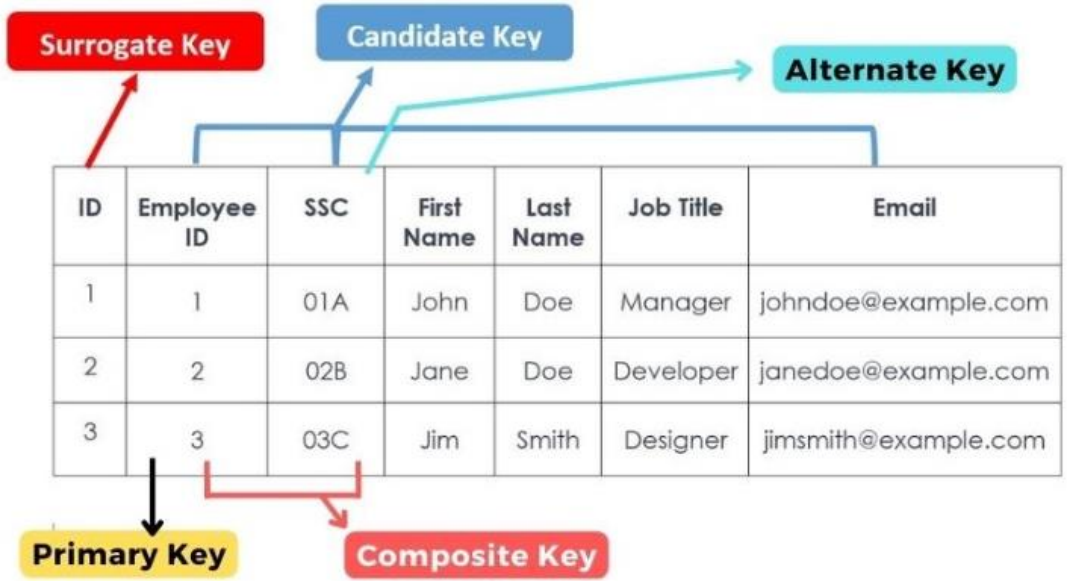
Example: Consider *Students* (Primary Key is StudentID), *Courses (*Primary Key is CourseID), and *Enrollments* relations, where a student can enroll in many courses, and a course has many students. *Enrollments* relation uses a combined primary key of (StudentID, CourseID), which is a compound key because StudentID is a foreign key referencing the *Students* table, and CourseID is a foreign key referencing the *Courses* table.

Key Types

Surrogate Key

- This is an artificially created value that uniquely identifies each entry in a table when no other column can hold properties of a primary key.
- It is an additional column and usually holds a generalized structure based on simple integers.

Example: Each branch has a natural key (EmployeeID) and also a surrogate key, namely ID.



ID	Employee ID	SSC	First Name	Last Name	Job Title	Email
1	1	01A	John	Doe	Manager	johndoe@example.com
2	2	02B	Jane	Doe	Developer	janedoe@example.com
3	3	03C	Jim	Smith	Designer	jimsmith@example.com

2.Functional Dependency

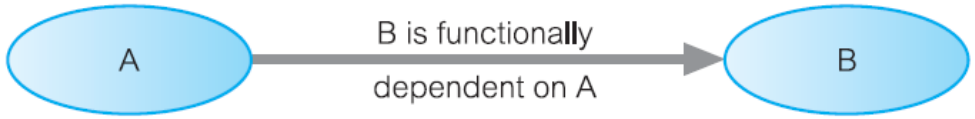
What is Functional Dependency?

- Consider a relational schema that has attributes (A, B, C, . . . , Z) and that the database is described by a single universal relation called R = (A, B, C, . . . , Z).
- **Functional Dependency (FD)** is a constraint that determines the relation of one attribute to another attribute in a DBMS.
- An FD is denoted by an arrow "→".

  Example: if A and B are attributes of relation R, **B is functionally dependent on A** (denoted A → B), if each value of A is associated with exactly one value of B.

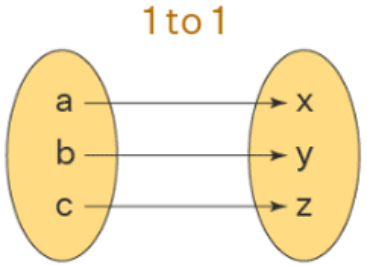
  (A and B may each consist of one or more attributes).

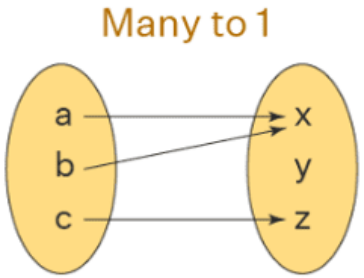
- An alternative way to describe the relationship between attributes A and B is to say that '**A functionally determines B**'.



What is Functional Dependency?

- There is a one-to-one relationship between the attribute(s) on the left-hand side and those on the right-hand side of a functional dependency.
  - The relationship in the opposite direction can be a one-to-one or one-to-many relationship.
- The attribute or group of attributes on the left-hand side of the arrow is called the determinant.





<u>Example</u>: Consider the functional dependency **A → B**

- Only one value of B in all the tuples that have a given value of A, at any moment in time.
- When two tuples have the same value of A, they also have the same value of B.
- However, for a given value of B, there may be several different values of A.

Types of functional dependencies

Types of functional dependencies in DBMS:

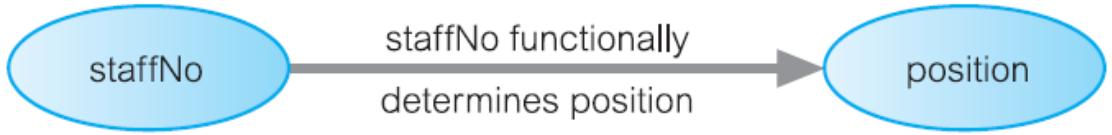
- Multivalued Dependency
- Trivial Functional Dependency
- Non-Trivial Functional Dependency
- Transitive Dependency

Apart from that, there are Fully Functional and Partial Functional dependencies.

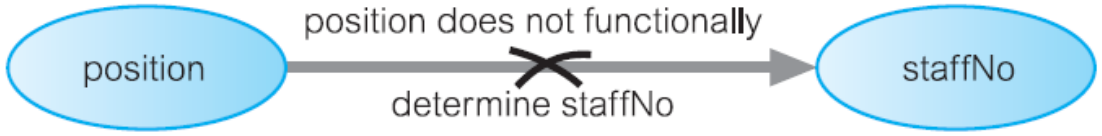
Example

Consider the attributes *staffNo* and *position* of the Staff relation.

- For a specific staffNo (**SL21**), you can determine the position of that member of staff as **Manager** (staffNo functionally determines position).



- However, the opposite is not true, as position does not functionally determine staffNo.



Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Example

- A member of staff holds one position; however, there may be several members of staff with the same position.

  Example: Staff numbers SL21 and SG5 are managers.

- The relationship between staffNo and position is one-to-one (1:1): for each staff number, there is only one position.
- The relationship between position and staffNo is one-to-many (1:*): there are several staff numbers associated with a given position.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Full functional dependency

- A full functional dependency occurs when you already meet the requirements for a functional dependency.
- Consider the functional dependency A → B, then B is fully functionally dependent on A, but not on any proper subset of A.
  - The determinants should have the minimal number of attributes necessary to maintain the FD with the attribute(s) on the right-hand side.
  - All non-key attributes depend on the key attribute.
  - The removal of any attribute from A results in the dependency no longer existing.
  - If there is a single determinant, it automatically becomes a full functional dependency.

Full functional dependency

<u>Example:</u>

Consider the functional dependency {staffNo, sName} → position exists in the *Staff* relation, but it is not a full functional dependency because you can remove sName from the left side of the statement without impacting the dependency relationship.

Activity

Consider the functional dependency staffNo, sName →branchNo that exists in the *Staff* relation. Is this a full functional dependency?

- Each value of (staffNo, sName) is associated with a single value of branchNo.
- However, it is not a full functional dependency because branchNo is also functionally dependent on a subset of (staffNo, sName), namely staffNo.
- The full functional dependency is staffNo →branchNo

Partial functional dependency

- A functional dependency A→B is a **partial dependency** if there is some attribute that can be removed from A and yet the dependency still holds.

Example:

{staffNo, sName} → position

{staffNo, sName} →branchNo

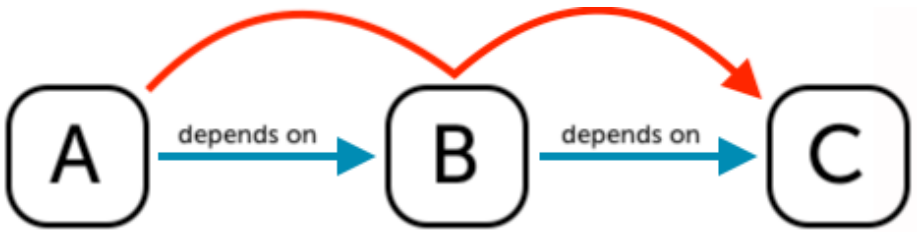
- A partial dependency is when you have a composite primary key, and non-key attributes functionally dependent on one, but not all of the attributes that make up the composite primary key.
- It is possible to derive full functional dependency from partial dependency.

staffNo → position

staffNo →branchNo

Transitive Dependency

- The condition where A, B, and C are attributes of a relation such that if A → B and B → C, then C is transitively dependent on A via B (provided that A is not functionally dependent on B or C).



- Transitive Dependency occurs because one non-key attribute is dependent on other non-key attributes.
- Transitive dependencies occur when there is an indirect relationship that causes a functional dependency.

Transitive Dependency

<u>Example</u>**:** Consider the following functional dependencies within the StaffBranch relation.

staffNo →sName, position, salary, branchNo, bAddress

branchNo →bAddress

- The transitive dependency branchNo → bAddress exists on staffNo via branchNo.
- In other words, the staffNo attribute functionally determines the bAddress via the branchNo attribute, and neither branchNo nor bAddress functionally determines staffNo.

Activity 1

Consider the attributes in the StaffBranch relation and identify the functional dependencies.

Assumption: The position held and the branch determine a member of staff's salary.

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

staffNo → sName, position, salary, branchNo, bAddress
branchNo → bAddress
bAddress → branchNo
branchNo, position → salary
bAddress, position → salary

Multi-Valued Dependency (MVD)

- Multi-Valued Dependency represents a dependency between attributes (for example, A, B, and C) in a relation, such that for each value of A, there is a set of values for B and a set of values for C.
- However, the sets of values for B and C are independent of each other.

Example: Consider the BranchStaffOwner relation, which displays the names of members of staff (sName) and property owners (oName) at each branch office (branchNo).

- Assume that sName uniquely identifies each member of staff and that the oName uniquely identifies each owner.
- Ann Beech and David Ford work at branch B003, and property owners called Carol Farrel and Tina Murphy are also registered at the same branch.
- However, there is no direct relationship between sName and oName at a given branch office.

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Multi-Valued Dependency (MVD)

- You must create a tuple for every combination of sName and oName to ensure that the relation is consistent.

  branchNo –>> sName

  branchNo –>> oName

- This constraint represents a multi-valued dependency in the BranchStaffOwner relation.

- Here, an MVD exists because two independent 1:* relationships are represented in the BranchStaffOwner relation.
- An MDV can be further defined as being **trivial** or **nontrivial**.

BranchStaffOwner

branchNo	sName	oName
B003	Ann Beech	Carol Farrel
B003	David Ford	Carol Farrel
B003	Ann Beech	Tina Murphy
B003	David Ford	Tina Murphy

Trivial and Nontrivial Dependencies

- An MVD A –>> B in relation R is defined as being trivial if,
  - **(a) B is a subset of A (B⊆A)** or
  - **(b) A ∪ B = R**
- A MVD is defined as being nontrivial if neither **(a)** nor **(b)** is satisfied.
- A trivial MVD does not specify a constraint on a relation, while a nontrivial MVD does specify a constraint.
- The MVD in the BranchStaffOwner relation is nontrivial as neither condition **(a)** nor **(b)** is true for this relation.
- The BranchStaffOwner relation is therefore constrained by the nontrivial MVD.

Trivial and Nontrivial Dependencies

Example: Add a new property owner for branch B003.

- You have to create two new tuples, one for each member of staff, to ensure that the relation remains consistent.
- This is an example of an update anomaly caused by the presence of the nontrivial MVD.
- The relation remains poorly structured, due to the data redundancy caused by the presence of the nontrivial MVD.

Activity 2 - Using sample data

Consider the Sample relation displaying data for attributes A, B, C, D, and E.

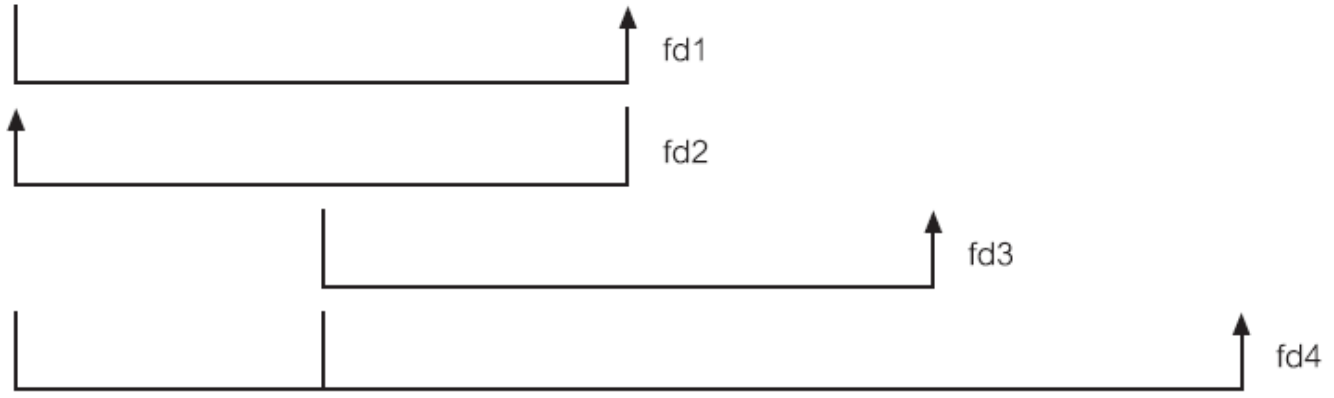
Identify the functional dependencies that exist between these attributes.

List down functional dependencies between attributes A to E in the Sample relation.

A → C (fd1)
C → A (fd2)
B → D (fd3)
A, B → E (fd4)

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t



Identifying the Primary Key for a Relation using Functional Dependencies

- Functional dependencies for a relation specify the set of integrity constraints that must hold on a relation.
  - An important integrity constraint to consider is the identification of candidate keys, one of which is selected to be the primary key for the relation.

Example 1: Identify the candidate key(s) for the StaffBranch relation using the functional dependencies identified in Activity 1.

determinants

staffNo → sName, position, salary, branchNo, bAddress
branchNo → bAddress
bAddress → branchNo
branchNo, position → salary
bAddress, position → salary

- The only candidate key and therefore the primary key is staffNo, as *all* other attributes of the relation are functionally dependent on staffNo.

Identifying the Primary Key for a Relation using Functional Dependencies

Example 2:

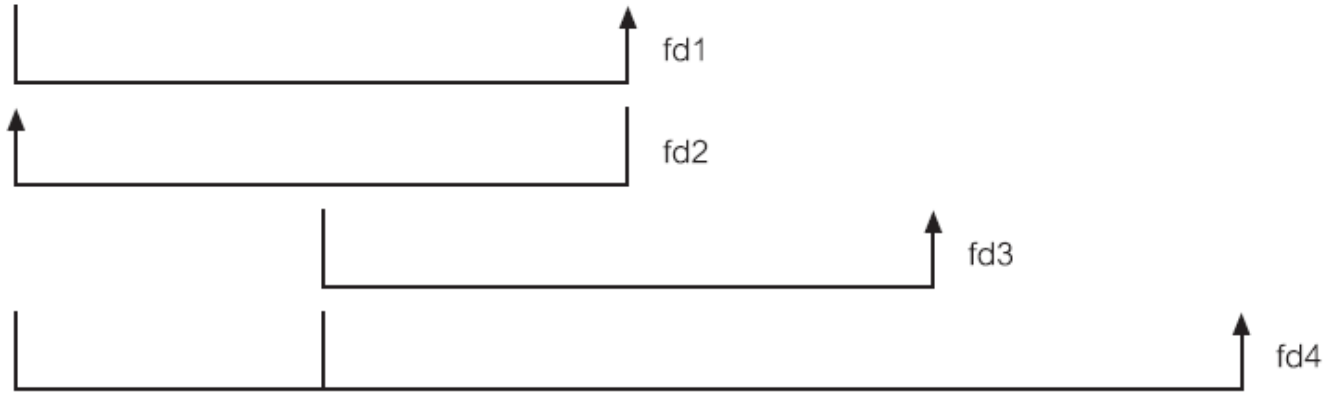
- Examine the determinant for each functional dependency to identify the candidate key(s) for the relation.

determinants

A → C (fd1)
C → A (fd2)
B → D (fd3)
A, B → E (fd4)

Sample Relation

A	B	C	D	E
a	b	z	w	q
e	b	r	w	p
a	d	z	w	t
e	d	r	w	q
a	f	z	s	t
e	f	r	s	t



Identifying the Primary Key for a Relation using Functional Dependencies

Example 2 (Cont..):

- The only determinant that functionally determines all the other attributes of the relation is (A,B).
  - A functionally determines C
  - B functionally determines D
  - (A, B) functionally determines E.
- So, the attributes that make up the determinant (A, B) can determine all the other attributes in the relation, either separately as A or B or together as (A, B).
- This is not a characteristic of the other determinants in the Sample relation as in each case they can determine only one other attribute in the relation.
- As there are no other candidate keys for the Sample relation **(A, B)** is identified as the primary key for this relation.

Wrap Up

- Key types (Super key, Candidate Key, Primary Key, Composite Key, Foreign Key, Compound Key, Surrogate Key)
- Functional Dependencies (Multivalued Dependency, Trivial Functional Dependency, Non-Trivial Functional Dependency, Transitive Dependency, Fully functional dependency, Partial dependency)

Any Questions?

In-class Assignment

- **Date:** 2nd January 2026
- **Venue:** 4th Floor (E401)
- **Time:** 1:00 PM – 2:15 PM: A tutorial session for Normalization
- **Time:** 2:30 PM – 3:00 PM: In-class assignment 1 from Normalization